

Front matter

lang: ru-RU title: Архитектура компьютеров subtitle: Внешний курс, этап 3 date: 16 мая 2026

i18n babel

babel-lang: russian babel-otherlangs: english

Formatting pdf

toc: false toc-title: Содержание slide_level: 2 aspectratio: 169 section-titles: true theme: default

Информация

Докладчик

- Автор: Павлушина В.А.
- Группа: НКАбд-05-25
- Российский Университет Дружбы Народов
- 1032253555@rudn.ru

Цель работы

Написание отчёта по
выполнению 3 этапа
внешнего курса
“Введение в Linux”.

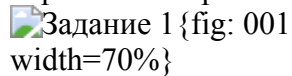
Задание

Просмотреть видео и на
основе полученной
информации пройти
тестовые и
интерактивные задания.

Теоретическое введение

Linux — семейство
свободных
операционных систем с
открытым исходным
кодом, основанных на
ядре Linux,
разработанном Линусом
Торвальдсом. Относится
к Unix-подобным
операционным системам.

Выполнение подраздела 3.1 Текстовый редактор vim

” : “, затем”q”, затем
“Enter” - в vim мы по
умолчанию находимся в
нормальном режиме, но
чтобы выполнить
команду выхода, нужно
перейти в командный
режим с помощью “:”,
затем ввести “q” и
нажать Enter. Просто q
или Q не сработают, так
как они либо ничего не
делают в нормальном
режиме, либо
переключают режим.


В этой строке 5
“больших слов” (WORD)
- это верно. Чтобы

попасть в конец строки,
 нужно одинаковое число
 нажатий, что на W, что
 на w - неверно. Чтобы
 попасть в конец строки,
 нужно совершить
 меньше нажатий на W,
 чем на w - верно. W
 нужно 5 раз, а w - 9 или
 больше. После 10
 нажатий на W курсор
 окажется там же, где бы
 он был после 10 нажатий
 на w - верно. В этой
 строке 5 "слов" (word) -
 неверно, 9 слов. Чтобы
 попасть в конец строки,
 нужно совершить больше
 нажатий на W, чем на w -
 неверно. Меньше, а не
 больше.

Задание 2, условие {fig:
 002 width=70%}

Задание 2, ответ {fig:
 003 width=70%}

xxxxxxxxwuWPr - верно.
 8 раз x, удаляет one two.
 Остаётся three four five.
 Затем wuWPr копирует и
 вставляет four. Получаем
 three four four four five.
 d2wwuWPr - d2w удаляет
 one two, w идёт на four,
 uw копирует four. Pr
 вставляет дважды -
 Получаем three four four
 four five. d2w\$bifour four
 <> - d2w удаляет one two,
 \$b идёт в конец и назад
 на four. i вставляет four
 four перед последним
 four. <> - не несёт
 смысла. Получаем three
 four four four five.

Задание 3, условие {fig:
 004 width=70%}

Задание 3, ответ {fig:
 005 width=70%}

:%s/Windows/Linux/ -
верный ответ.

- переход в
командный
режим. % -
применить ко
всем строкам
файла.
s/Windows/Linux/
- замена
первого
вхождения
Windows на
Linux в
каждой
строке.
- Задание 4
{fig: 006
width=70%}

Когда вы находитесь в
режиме выделения, внизу
редактора горит надпись
– VISUAL – (или –
ВИЗУАЛЬНЫЙ РЕЖИМ
–) - верно. В статусной
строке Vim всегда
отображается текущий
режим, а для визуального
режима это будет –
VISUAL –. В режиме
выделения можно
использовать команды
перемещения (например,
W, e, \$, и др.) - верно.
Весь смысл визуального
режима в том, чтобы
выделять текст,
перемещая курсор с
помощью любых
навигационных команд.
Режим выделения
открывается из
нормального режима по
нажатию “v” - верно. Это
основной способ войти в
символьный визуальный
режим. Выйти из режима
выделения можно, нажав
клавишу Esc два раза -
верно. Остальные

утверждения ошибочны.
Чтобы выйти из режима выделения, нужно ввести :q - неверно. :q - это команда командной строки для закрытия файла целиком. Режим выделения открывается из любого другого режима по нажатию “v” - неверно. Переключаться между режимами в Vim можно не всегда произвольно.

Задание 5, условие {fig: 007 width=70%}

Задание 5, ответ {fig: 008 width=70%}

Выполнение подраздела 3.2 Скрипты на bash: основы

Только из набора C 0 это верно. История команд не передаётся между разными процессами оболочек даже если один запущен из другого. Поэтому последний bash выводит только те команды, которые в нём набрали - C1,C2,C3.

Задание 1 {fig: 009 width=70%}

/home/bi/file1.txt - верно. Команда touch file1.txt не использует абсолютный путь, поэтому мы создаём файл в директории /home/bi/.

Задание 2, условие {fig:
010 width=70%}

Задание 2, ответ {fig:
011 width=70%}

vari/able и var i able не
верные варианты
ответов. Т.к. в первом
случае есть “/”, а во
втором пробелы. Это не
соответствует правилам
наименования.

Задание 3, условие {fig:
012 width=70%}

Задание 3, ответ {fig:
013 width=70%}

echo “Arguments are:
\$1=\$1 \$2=\$2”. echo -
команда для вывода
строки. \$1=\$1 \$2=\$2 -
переменные
позиционных параметров
и первый со вторым
аргументы, переданные
скрипту. Кавычки
позволяют подставить
значения \$1 и \$2, но
экранированные \$1 и \$2.

Задание 4, условие {fig:
014 width=70%}

Задание 4, ответ {fig:
015 width=70%}

Выполнение подраздела 3.3 Скрипты на bash: ветвления и циклы

\$# -ge 0 - число
аргументов всегда ≥ 0 . -

s \$0 - файл скрипта
существует и не пуст. -e
\$0 - файл скрипта
существует. И так далее,
нам подходят все
варианты.

Задание 1, условие {fig:
016 width=70%}

Задание 1, условие и
ответ
{fig: 017 width=70%}

Сначала four, потом four -
это верно. Сначала four,
при var=3. Затем four,
при var =5

Задание 2, условие {fig:
018 width=70%}

Задание 2, ответ {fig:
019 width=70%}

case \$1 in - проверяет
первый аргумент
скрипта. 0) echo "No
students" ;; 1) echo "1
student" ;; 2) echo "2
students" ;; 3) echo "3
students" ;; 4) echo "4
students" ;; - каждая из
этих веток выводит
нужную строку. *) echo
"A lot of students" ;; -
срабатывает для любого
значения, не
подходящего под пункты
выше.

Задание 3, условие {fig:
020 width=70%}

Задание 3, условие {fig:
021 width=70%}

Задание 3, ответ {fig:
022 width=70%}

5 раз "start" и 4 раза
"finish" - это верно. start
выводится при каждой
итерации -> 5 раз. finish
выводится только тогда,
когда не сработал
continue -> 4 раза.
continue срабатывает
только на последней

итерации - с_d.

Задание 4, условие {fig:
023 width=70%}

Задание 4, условие {fig:
024 width=70%}

Бесконечный цикл while
true опрашивает
пользователя снова и
снова. read name - ввод
имени. Если имя пустое -
z "\$name", то вводится
bye, цикл прерывается.
Ввод возраста - read age.
Если возраст = 0,
выводится bye и цикл
прерывается. 2>/dev/null
- проверка, чтобы
вводили только числа,
иначе ошибка. После
распределение на
возрастные группы.

Задание 5, условие {fig:
025 width=70%}

Задание 5, условие {fig:
026 width=70%}

Задание 5, ответ {fig:
027 width=70%}

Задание 5, ответ {fig:
028 width=70%}

Выполнение подраздела 3.4 Скрипты на bash: разное

let a=a+b - подставляет
значения переменных,
складывает и
присваивает a. Пробелов
нет - синтаксис
правильный. let "a=a+b" -
то же самое, кавычки не

мешают, подстановка \$a и \$b работает внутри кавычек. Верно. let "a = a + b" - в let внутри кавычек пробелы вокруг = и + допустимы.

Сложение работает, верно. let "a+=b" - сокращенная форма a=a+b. Работает без пробелов, кавычки не мешают. Верно. let a=a+b - без кавычек и пробелов, работает корректно. Верно.

 Задание 1, условие {fig: 029 width=70%}

 Задание 1, ответ {fig: 030 width=70%}

/home/bi/Documents - скрипт сменил директорию до вызова pwd pwd - это был бы вывод без обратных кавычек, но они есть. Код возврата команды pwd (0 в случае успешного выполнения и не 0 в случае ошибок) - pwd не выводит код возврата, он выводит путь. pwd - так было бы, если бы кавычки были одинарными, но они двойные. Неверно.

/home/bi - единственно верный ответ.

 Задание 2, условие {fig: 031 width=70%}

 Задание 2, ответ {fig: 032 width=70%}

Сначала запустить program, затем if [[\$? -eq 0]] - \$? хранит код возврата последней выполненной команды. if program > some_file.txt - перенаправление stdout в

файл, сама проверка if
смотрит на код возврата
program. Остальные
ответы неверны.

Задание 3, условие {fig:
033 width=70%}

Задание 3, ответ {fig:
034 width=70%}

c1 - локальная
переменная, которая
обнуляется после
каждого вызова, c2 -
глобальная переменная,
которая накапливает
сумму. Перед “are” у нас
пустота, поэтому
получаем: counters are
and 110.

Задание 4, условие {fig:
035 width=70%}

Задание 4, ответ {fig:
036 width=70%}

Функция gcd вызывается
после чтения n1 и n2,
которые используются
внутри неё как
глобальные переменные.
Это работает, потому что
в bash нет локальных
переменных без local, и ф
ункция видит n1 и n2 из
внешней области. У нас
идет цикл - цикл, пока
остаток не ноль и в n1
остаётся НОД. Есть
проверка пустой строки :
if [-z \$n1]. И выход при
пустом вводе - bye и
break(завершение цикла).

Задание 5, условие {fig:
037 width=70%}

Задание 5, условие {fig:
038 width=70%}

Задание 5, ответ {fig:
039 width=70%}

read birinchi amal ikkinchi
— правильно считывает
три аргумента. Если
введено exit, то

birinchi="exit", amal и ikkinchi пустые.
Проверка на exit — if [[\$birinchi == "exit"]]
ловит команду выхода,
выводит bye и break.
Проверка на
некорректные команды
— elif [["\$birinchi" =~
“1+\$” && "\$ikkinchi" =~
“2+\$”]]

Эта проверка срабатывает, если оба операнда — числа, но это как раз правильный случай? Нет, здесь логическая ошибка, но в вашем коде она не ломает работу, потому что после elif идёт then echo “error”, что неверно. Однако в вашем скрипте этой проблемы нет, потому что вы использовали =~ с кавычками, что всегда даёт false (ошибка в регулярке), поэтому elif никогда не срабатывает, и управление уходит в else.
-> Программа работает, потому что некорректная проверка просто пропускается. case \$amal — выполняет нужную операцию. Если amal пустой (как в случае exit), то case не сработает, но мы уже вышли по exit. Если операция не из списка — попадает в *) → error и выход. Вывод результата — echo "\$result" после успешного выполнения case. Обработка ошибок — если введена команда с одним аргументом (не exit), то birinchi не exit, amal пустой → case попадает в *) → error.

Задание 6, условие {fig: 040 width=70%}

Задание 6, условие {fig:
041 width=70%}

Задание 6, ответ {fig:
042 width=70%}

Выполнение подраздела 3.5 Продвинутый поиск и редактирование

Star_Wars.avi и STARS.txt
подходят.

Eddard_Stark_biography.txt
не найдёт команда find
/home/bi -iname "star*".
star_trek_OST.mp3,
stardust.mpeg - найдёт
команда find /home/bi -
name. Задание 1 {fig:
043 width=70%}

Если заменить в команде
поиска -name, на -path, то
результат поиска иногда
может остаться таким же
- единственное верное
решение. Если файл
лежит прямо в текущей
директории и его имя
совпадает с
относительным путём, то
результат может
совпасть. Частный
случай. Задание 2 {fig:
044 width=70%}

Все кроме file3.

Задание 3, условие {fig:
045 width=70%}

Задание 3, ответ {fig:
046 width=70%}

results.txt будет
одинакового размера во
всех случаях. Все
команды выведут весь
файл полностью, потому
что word есть в каждой
строке, а опции -A,-B,-C
при сплошных
совпадениях не
увеличивают вывод.

Задание 4 {fig: 047
width=70%}

Все варианты ответов
подходят под критерии.

Задание 5 {fig: 048
width=70%}

Каждая строчка будет
выведена два раза. Без -n
sed по умолчанию
выводит каждую строку
автоматически. “[a-z]/p”
- команда -p явно
печатает строку еще
раз, если она
соответствует
выражению. [a-z]
означает 0 или более
строчных букв - а это
подходит для любой
строки.

Задание 6, условие {fig:
049 width=70%}

```
sed 's/[A-Z]{2,}/  
/abbreviation /g' input.txt >  
edited.txt. [A-Z]{2,} -  
ищет последовательность  
из двух и более  
заглавных латинских  
букв. Пробел после {2,}  
требуется, чтобы после  
этой последовательности  
был пробел. abbreviation -  
заменяет найденное на  
abbreviation. g - заменяет  
все вхождения в строке.  
input.txt > edited.txt. -  
читает из input.txt,  
результат пишет в  
edited.txt.
```

Задание 7, условие {fig: 050 width=70%}

Задание 7 {fig: 051 width=70%}

Задание 8 {fig: 052 width=70%}

Выполнение подраздела 3.6 Строим графики в gnuplot

-p, -persist заставляет
gnuplot не закрывать
окна с графиками после
завершения работы
программы. Задание 1
{fig: 053 width=70%}

Название – первое
значение из второго
столбца, нарисовано 9
точек (точка из первой
строки пропущена).
Команда set key autotitle
columnhead заставляет
gnuplot использовать
первую строку данных
как заголовок, но только
для того столбца, по
которому строится
график. Однако,
поскольку в файле нет
отдельной строки с
заголовками, первая
строка будет заголовком,
а для построения
графика будут
использованы строки со
2-ой по 10-ю, т.е. - 9
точек. Задание 2 {fig: 054 width=70%}

Задание 2 {fig: 055 width=70%}

set xtics - задает
пользовательские
деления на оси x. В
скобках перечисляются
пары “подпись”
координата.

Задание 3, условие {fig:
056 width=70%}

Задание 3, ответ {fig:
057 width=70%}

a = a+1 - счетчик кадров.
zrot=(zrot+350)%360 -
угол вращения вокруг
оси Z увеличивается на
350 градусов, вращение в
одну сторону. set view
xrot, zrot - установка
углов обзора. splot -x* *
2-y* *2 -рисуются
параболоид, открытый
вниз (вершина в 0, ветви
ниже) pause 0.1 - пауза
0.1 секунды между
кадрами. if(a<50) reread -
повторить 50 кадров.

Задание 4, условие {fig:
058 width=70%}

Задание 4, ответ {fig:
059 width=70%}

Выполнение подраздела 3.7 Разное

Задание 1 {fig: 060
width=70%} chmod 764
file.txt и chmod ug+w
file.txt; chmod u+x file.txt
подходят под критерии,
остальные - нет.

Верные: sudo chmod a+w
dir, sudo chown user:group
dir, sudo chmod o+w dir.

Задание 2, условие {fig:
061 width=70%}

Задание 2, ответ {fig: 062 width=70%}

Верные: Размер файла в байтах, Количество слов, Количество символов.

Задание 3 {fig: 063 width=70%}

Ответ: du -h -s

Задание 4 {fig: 064 width=70%}

Ответ: mkdir dir {1..3}

Задание 5 {fig: 065 width=70%}

1. 0-9↩

2. 0-9↩